INFORME PRELIMINAR DESAFIO 1 IMPLEMENTACION DE TETRIS EN C++

INTEGRANTES:

ISABELLA CARVAJAL

CC.1094903251

JUAN SOLARTE

CC.1081052976

CURSO:

INFORMATICA II

UNIVERSIDAD DE ANTIOQUIA

SEMESTRE 2026-1

MEDELLIN-ANTIOQUIA

13/03/2026

Desafío I

Implementación de Tetris mediante manipulación de bits en C++

Tabla de contenido

- Contextualización
- Análisis del problema
- Diseño del sistema
- Arquitectura del programa
- Algoritmos principales
- Problemas encontrados durante el desarrollo
- Evolución de la solución

1. CONTEXTUALIZACION:

El presente desafío consiste en desarrollar una versión simplificada del juego Tetris utilizando el lenguaje de programación C++. El sistema funcionará por consola y permitirá al usuario interactuar con el juego mediante comandos para mover y rotar piezas. A diferencia del Tetris tradicional, esta implementación no utiliza gráficos complejos ni animaciones en tiempo real. En su lugar, el estado del tablero se mostrará mediante caracteres en consola. El documento del reto exige precisamente una interfaz por consola, sin animaciones en tiempo real, donde el sistema muestre el tablero y procese la acción elegida por el usuario en cada turno.

El objetivo principal del proyecto es aplicar varios conceptos estudiados durante el curso de programación, como operadores a nivel de bits, manejo de memoria dinámica, uso de punteros, arreglos y programación modular. Además, se deben respetar restricciones importantes como no usar STL, no usar string y realizar la implementación en C++ dentro de Qt. Estas condiciones hacen que la solución no solo deba funcionar, sino que también esté bien pensada desde el punto de vista de la eficiencia y de la estructura del programa.

La idea general de la solución es representar tanto el tablero como las piezas mediante bits. Esto permite aprovechar operaciones como desplazamientos y máscaras para mover piezas, detectar colisiones y revisar si una fila está completa.

2. ANÁLISIS DEL PROBLEMA

Para desarrollar correctamente el juego es necesario dividir el problema en varias partes: representación del tablero, representación de las piezas, movimiento, rotación, detección de colisiones, eliminación de filas y condición de finalización.

2.1. REPRESENTACIÓN DE LAS PIEZAS

Las piezas del Tetris pueden representarse mediante pequeños patrones binarios. El enunciado menciona los siete tetriminos clásicos: I, O, T, S, Z, J y L. Además, se pide implementar al menos cinco tipos de piezas y generar las figuras de manera aleatoria uniforme desde la ubicación central en el ancho del tablero.

Por ejemplo, una pieza T se puede pensar así:

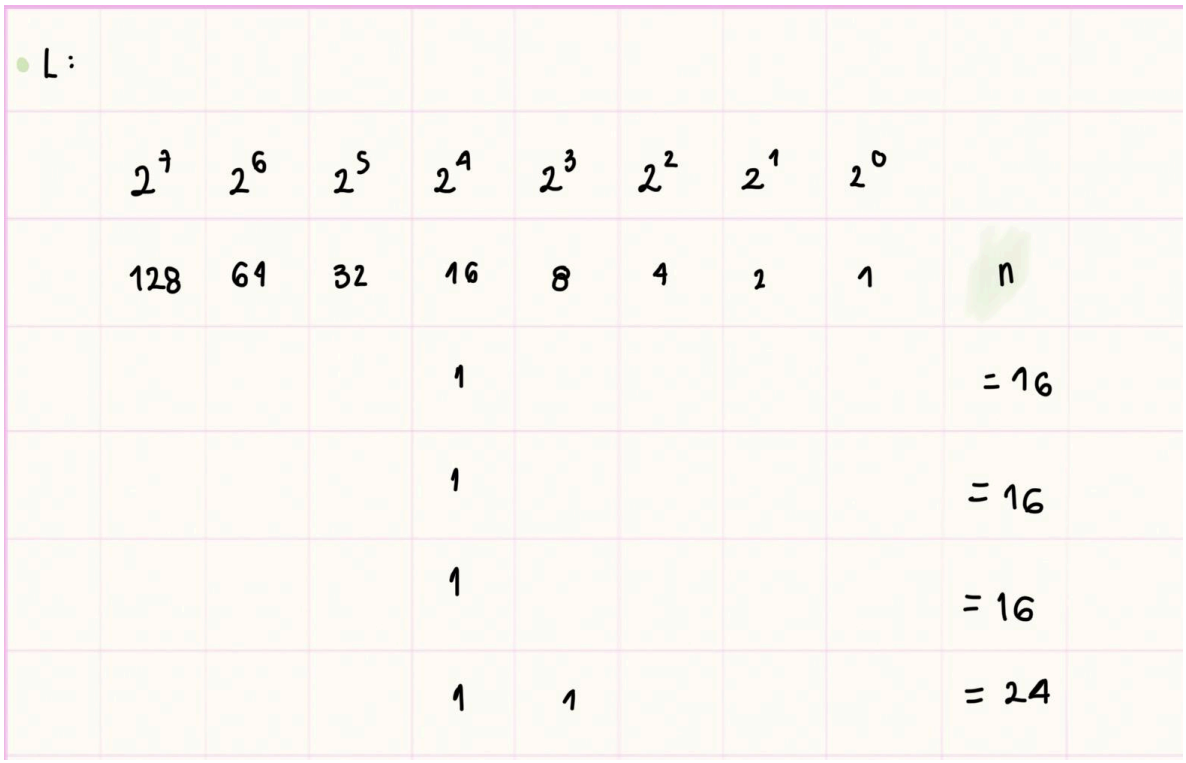
00011100

00001000

00001000

La rotación de las piezas ya que no se pueden almacenar, serán usando máscaras definidas para cada pieza. Es decir, cada pieza tendrá su definición respectiva de máscaras para las rotaciones que se harán conforme la ejecución del juego.

Se piensa guardar las piezas en un arreglo estático bidimensional, ya que las fichas no cambiarán su forma durante la ejecución, cambiarán la manera en la que estarán que se hablará más adelante. Se habla de un arreglo estático donde la primera dimensión almacenará las 5 piezas que se han de seleccionar, en la segunda dimensión se plantea representar la forma decimal del número binario. La imagen muestra la representación decimal que se piensa de la pieza L. Donde el arreglo de esa pieza en particular se podría ver algo así: [16,16,16,24]. Cabe recalcar que los espacios vacíos en el dibujo, representan ceros.



2.2. *REPRESENTACION DEL TABLERO*

El tablero debe tener un ancho y un alto definidos por el usuario. Según el enunciado, ambas dimensiones deben ser mínimo de 8 y el ancho debe ser estrictamente múltiplo de 8. Esta condición es importante porque facilita la representación del tablero usando bytes, donde cada grupo de 8 bits puede representar 8 celdas del tablero.

Cada bit tendrá un significado simple:

- 1 representa una celda ocupada
- 0 representa una celda vacía

Un ejemplo conceptual de una fila del tablero podría verse así:

00000000
00111000
00101000
00111000

Como el tamaño del tablero se define durante la ejecución, no conviene usar tamaños fijos. Por esa razón, se utilizará memoria dinámica para crear el tablero. Esto además cumple con uno de los requisitos obligatorios del desafío, que exige el uso de punteros, arreglos y memoria dinámica. Para la representación de este tablero se usara un arreglo dinámico bidimensional.

2.3. MOVIMIENTO DE LAS PIEZAS

En cada turno el usuario puede elegir una de estas acciones:

- mover una posición a la izquierda
- mover una posición a la derecha
- bajar una posición
- rotar la pieza

Eso está definido de forma explícita en el documento del desafío.

Para desplazar horizontalmente las piezas se pueden utilizar operaciones de desplazamiento de bits (<<,>>:). Por ejemplo, mover una figura a la izquierda puede interpretarse como desplazar sus bits en una dirección, y moverla a la derecha implica desplazarlos en la dirección contraria. Sin embargo, no basta con mover los bits: antes de aceptar el movimiento hay que revisar que la pieza no salga de los límites del tablero y que no choque con bloques ya ocupados que se retomara este problema en la sección de colisiones.

Para la rotación de las figuras se propone la utilización de máscaras las cuales variarán respecto a la pieza que se tenga en el momento.

2.4. INTERFAZ POR CONSOLA

Para la interfaz, íntimamente relacionada con el tablero, se plantea la impresión de las piezas de acuerdo a su representación binaria, si estas han de rotar se trabajará con las máscaras y punteros que apunten hacia la pieza en el arreglo (es decir sus valores decimales representativos) para que la estructura de la pieza no cambie, solo cambie su representación desde la operatividad y su impresión en pantalla.

Cada vez que el tablero cambie, el programa debe imprimir una representación visual en consola, por ejemplo, usando # (representando un 1) para los bloques ocupados y _ (Representando un 0) para los espacios vacíos, relacionado con los valores discutidos

anteriormente de las piezas. También debe indicar claramente cuándo ocurre un Game Over.

Por lo tanto, la visualización no necesita estructuras complejas. Basta con recorrer el tablero y mostrar un símbolo diferente según el valor de cada bit. Esta solución es simple, clara y cumple con lo solicitado.

2.4. DETECCIÓN DE COLISIONES

La detección de colisiones es una de las partes más importantes del proyecto. El enunciado indica que se debe verificar si un movimiento es válido comparando los bits de la pieza con los bits ocupados en el tablero, teniendo en cuenta también las fronteras. Para esta detección de colisiones, dependerá del tipo de pieza para garantizar que la cantidad de líneas a analizar sea la correcta.

2.5. ELIMINACIÓN DE FILAS

Cuando una fila del tablero tenga todos sus bits en 1, debe eliminarse y las filas superiores deben bajar una posición. El mismo enunciado aclara que esto debe hacerse usando manipulación de punteros y bits.

Para detectar si una fila está completa, se compara su patrón con el valor que representa todos los bits encendidos en el ancho útil del tablero. Si la fila está llena, se elimina traduciéndose a liberar la memoria dinámica que ocupa la debida fila, se desplazan las filas superiores hacia abajo y se crea una nueva fila vacía en la parte superior.

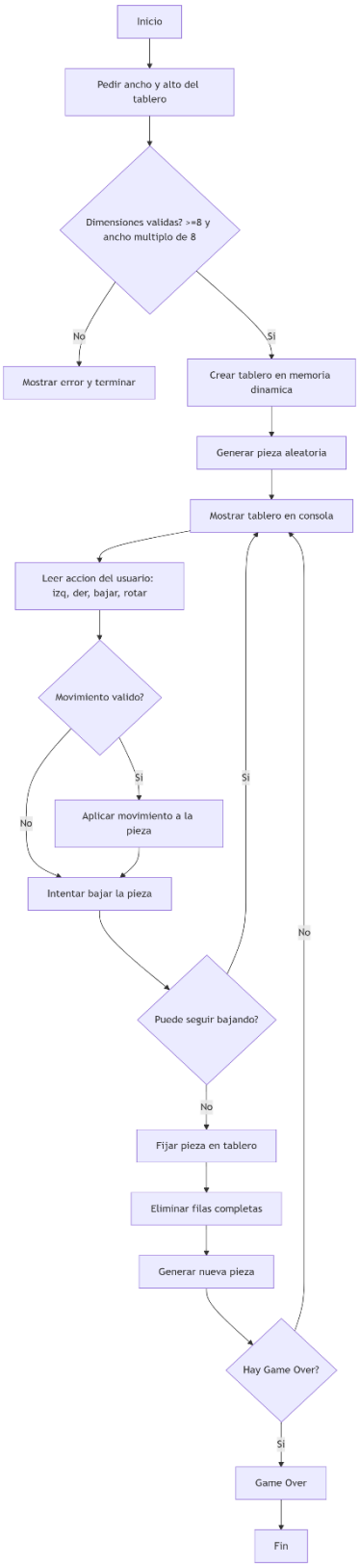
2.7. CONDICION DE GAME OVER

El juego termina cuando una nueva pieza no puede colocarse en la parte superior del tablero. Esa es la condición de Game Over.

En otras palabras, si al generar una nueva pieza ya existe colisión desde la posición inicial, entonces el programa debe terminar e informar que el juego ha finalizado.

4. ARQUITECTURA DEL PROGRAMA

La estructura general del sistema puede resumirse de la siguiente manera:



Código Mermaid (mejor visualización)

```
flowchart TD

A[Inicio] --> B[Pedir ancho y alto del tablero]

B --> C{Dimensiones validas? >=8 y ancho multiplo de 8}

C -- No --> D[Mostrar error y terminar]

C -- Si --> E[Crear tablero en memoria dinamica]

E --> F[Generar pieza aleatoria]

F --> G[Mostrar tablero en consola]

G --> H[Leer accion del usuario: izq, der, bajar, rotar]

H --> I{Movimiento valido?}

I -- Si --> J[Aplicar movimiento a la pieza]

I -- No --> K[Intentar bajar la pieza]

J --> K

K --> L{Puede seguir bajando?}

L -- Si --> G

L -- No --> M[Fijar pieza en tablero]

M --> N[Eliminar filas completas]

N --> O[Generar nueva pieza]

O --> P{Hay Game Over?}

P -- Si --> Q[Game Over]

Q --> R[Fin]

P -- No --> G
```

5. ALGORITMOS PRINCIPALES.

En esta sección se describen los algoritmos principales que permiten el funcionamiento del sistema. Cada algoritmo representa una de las operaciones fundamentales del juego, como la inicialización del tablero, la generación de piezas, el movimiento de las mismas y la eliminación de filas completas.

Para facilitar la comprensión del funcionamiento del sistema, los algoritmos se presentan utilizando una secuencia de pasos conectados mediante flechas que representan el flujo de ejecución.

5.1. ALGORITMO DE INICIALIZACION DEL TABLERO

Este algoritmo se encarga de crear el tablero del juego a partir de las dimensiones ingresadas por el usuario.

Inicio
↓
Solicitar ancho del tablero
↓
Solicitar alto del tablero
↓
Verificar que las dimensiones sean mayores o iguales a 8

→ No
↓
Mostrar mensaje de error
↓
Finalizar programa

→ Sí
↓
Verificar que el ancho sea múltiplo de 8

→ No
↓
Mostrar mensaje de error
↓
Finalizar programa

→ Sí
↓
Reservar memoria dinámica para el tablero
↓
Inicializar todas las posiciones del tablero en 0
↓
Fin

5.2. ALGORTIMO DE GENERACION DE PIEZA

Este algoritmo se encarga de generar una nueva pieza del juego de forma aleatoria.

Inicio
↓
Seleccionar aleatoriamente un tipo de pieza
↓
Asignar la pieza seleccionada
↓
Asignar rotación inicial
↓
Ubicar la pieza en la parte superior del tablero
↓
Ubicar la pieza en el centro horizontal del tablero
↓
Fin

5.3 ALGORITMO DE DETECCION DE COLISION

Este algoritmo verifica si una pieza en una posición determinada entra en conflicto con bloques ya existentes en el tablero.

Inicio
↓
Tomar la posición actual o propuesta de la pieza
↓
Recorrer cada fila de la pieza
↓
Comparar los bits de la pieza con los bits ocupados del tablero
↓
¿Existe superposición de bits?

→ Sí
↓
Existe colisión
↓
Fin

→ No
↓
¿Se revisaron todas las filas de la pieza?

→ No
↓
Continuar con la siguiente fila

→ Sí
↓
No existe colisión → Fin.

5.4 ALGORITMO DE MOVIMIENTO DE LA PIEZA

Este algoritmo gestiona las acciones que el usuario puede realizar sobre la pieza actual.

Inicio
↓
Leer la acción ingresada por el usuario
↓
Determinar el tipo de movimiento

• mover a la izquierda
• mover a la derecha
• bajar pieza
• rotar pieza

↓
Calcular nueva posición o rotación
↓
Verificar si el cambio produce colisión
↓
¿Existe colisión?

→ Sí
↓
Cancelar movimiento
↓
Fin

→ No
↓
Actualizar posición o rotación de la pieza
↓
Fin

5.5. ALGORITMO DE FIJACION DE LA PIEZA

Este algoritmo se ejecuta cuando una pieza ya no puede seguir descendiendo.

Inicio
↓
Tomar la posición actual de la pieza
↓
Recorrer cada fila de la pieza
↓
Combinar los bits de la pieza con los bits del tablero
↓
Actualizar las posiciones correspondientes del tablero

↓
¿Se procesaron todas las filas de la pieza?

→ No
↓
Continuar con la siguiente fila

→ Sí
↓
Fin

5.6. ALGORITMO DE ELIMINACION DE FILAS COMPLETAS

Este algoritmo detecta y elimina las filas que se encuentren completamente ocupadas.

Inicio
↓
Recorrer las filas del tablero
↓
Verificar si la fila está completamente llena
↓
¿La fila está llena?

→ No
↓
Continuar con la siguiente fila

→ Sí
↓
Eliminar la fila
↓
Desplazar hacia abajo las filas superiores
↓
Crear una nueva fila vacía en la parte superior
↓
Continuar revisión del tablero

↓
¿Se revisaron todas las filas?

→ No
↓
Seguir revisando

→ Sí
↓
Fin

5.7 ALGORITMO PRINCIPAL DEL JUEGO

Este algoritmo representa el flujo general del sistema.

Inicio
↓
Inicializar tablero
↓
Generar primera pieza
↓
Mostrar tablero
↓
Leer acción del usuario
↓
Intentar mover o rotar la pieza
↓
Intentar bajar la pieza
↓
¿La pieza puede seguir bajando?

→ Sí
↓
Volver a mostrar el tablero

→ No
↓
Fijar pieza en el tablero
↓
Eliminar filas completas
↓
Generar nueva pieza
↓
¿La nueva pieza puede colocarse?

→ No
↓
Mostrar mensaje de Game Over
↓
Fin

→ Sí
↓
Volver a mostrar el tablero

5.8 RESUMEN DEL FUNCIONAMIENTO DEL SISTEMA

El funcionamiento general del juego puede resumirse de la siguiente manera:

Inicializar tablero
↓
Generar pieza
↓
Mostrar tablero
↓
Leer acción del usuario
↓
Mover o rotar pieza
↓
Verificar colisiones
↓
Intentar descenso de la pieza
↓
¿Puede continuar bajando?

→ Sí → Continuar ciclo

→ No
↓
Fijar pieza en tablero
↓
Eliminar filas completas
↓
Generar nueva pieza
↓
Verificar Game Over
↓
Repetir ciclo

6. PROBLEMAS ENCONTRADOS DURANTE EL DESARROLLO

Durante el proceso de análisis y diseño de la solución se identificaron varios desafíos relacionados con la representación eficiente de los elementos del juego y con la manipulación de bits para controlar el comportamiento de las piezas.

Uno de los principales problemas fue definir una forma adecuada de representar el tablero. Debido a que el ancho del tablero debe ser múltiplo de 8, se decidió utilizar una representación basada en bits, donde cada bit corresponde a una celda del tablero. Esto permite aprovechar mejor la memoria y facilita operaciones como la detección de colisiones y la eliminación de filas completas. Sin embargo, trabajar directamente con bits requiere un mayor cuidado al momento de realizar desplazamientos o comparaciones.

Otro problema identificado fue la representación de las piezas del Tetris. Cada pieza puede tener varias rotaciones posibles, por lo que fue necesario decidir si calcular las rotaciones dinámicamente o almacenarlas previamente. Se optó por definir las rotaciones dependientes a las figuras trabajando con punteros que garantizarán eficiencia a la hora de trabajar con arreglos.

También se identificó como dificultad el manejo de los movimientos de las piezas dentro del tablero. Cada vez que el usuario realiza una acción, el programa debe verificar que la nueva posición no genere colisiones ni salga de los límites del tablero. Esto implica comparar continuamente los bits de la pieza con los bits ya ocupados en el tablero.

Finalmente, otro aspecto que requiere especial cuidado es la eliminación de filas completas. Cuando una fila está llena, se debe eliminar y desplazar las filas superiores hacia abajo. Este proceso debe realizarse correctamente utilizando memoria dinámica y punteros para evitar perder información o generar inconsistencias en el tablero.

7. EVOLUCIÓN DE LA SOLUCIÓN.

El desarrollo de la solución se planteó de forma progresiva, comenzando con el análisis del problema y la identificación de los elementos principales del sistema.

En una primera etapa se analizó la estructura general del juego y se definieron los componentes básicos: tablero, piezas, movimientos y reglas del juego. En esta etapa se consideraron diferentes alternativas para representar el tablero y las piezas.

Posteriormente, se decidió utilizar una representación basada en bits, ya que esta permite trabajar de forma eficiente con las operaciones necesarias para el funcionamiento del juego. Esta decisión también se ajusta a los objetivos del desafío, que buscan evaluar el uso de operadores a nivel de bits.

En la siguiente etapa se definieron los algoritmos principales del sistema, incluyendo la generación de piezas, la detección de colisiones, el movimiento de las piezas y la eliminación de filas completas. Estos algoritmos se organizaron de forma modular para facilitar la implementación del programa.

También se diseñó la arquitectura general del sistema mediante diagramas de flujo que representan el funcionamiento del juego paso a paso. Esto permitió visualizar de manera clara la secuencia de acciones que realiza el programa durante cada turno.

Finalmente, se estructuró la solución de manera que cumpla con las restricciones establecidas en el desafío, incluyendo el uso de memoria dinámica, punteros y manipulación de bits, así como la organización del código en módulos separados.

La solución propuesta constituye una base sólida para la etapa de implementación del programa, en la cual se desarrollará el código fuente que permita ejecutar el juego y demostrar el correcto funcionamiento de todos los algoritmos definidos.